

## Лабораторная работа № 5

### Шифрование с открытым ключом. Алгоритм RSA

#### Вариант №15

Ф. И. О. студента: Рау Алексей Евгеньевич

Группа: ФИТ-231

Проверил: Белим С.Ю.

Дата: 11.11.2025

#### Основные сведения

Факторизация числа  $n = p * q$

Функция Эйлера:  $\varphi(n) = (p - 1) * (q - 1)$

Формула для нахождения закрытого ключа по известному открытому ключу:  $d = e^{-1} \bmod \varphi(n)$

#### Результаты

Параметр  $n = 54053$

Открытый ключ шифрования  $e = 251$

Факторизация числа  $n$ :  $p = 191, q = 283$

Закрытый ключ шифрования  $d = 28391$

Шифр-текст: 13047

Открытый текст: 2315

#### Код программы

```
import math

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(e, phi):
    gcd, x, y = extended_gcd(e, phi)
    if gcd != 1:
        raise ValueError("Обратный элемент не существует")
    return x % phi
```

```

def bin_pow(a, k, n):
    b = 1
    a = a % n
    while k > 0:
        if k % 2 == 0:
            k //= 2
            a = (a * a) % n
        else:
            k -= 1
            b = (b * a) % n
    return b

def sieve_primes(limit):
    is_prime = [True] * (limit + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(limit**0.5) + 1):
        if is_prime[i]:
            for j in range(i * i, limit + 1, i):
                is_prime[j] = False
    primes = [i for i in range(2, limit + 1) if is_prime[i]]
    return primes

def extract_powers(n, divisor):
    count = 0
    while n % divisor == 0:
        n //= divisor
        count += 1
    return n, count

def trial_division_factorization(n):
    original_n = n
    factors = []

    for p in [2, 3]:
        n, cnt = extract_powers(n, p)
        if cnt > 0:
            factors.extend([p] * cnt)

    if n == 1:
        return factors

    limit = int(math.isqrt(n)) + 1
    primes = sieve_primes(limit)

    i = 0
    while i < len(primes):
        qs = 1
        j = i
        while j < len(primes) and j < i + 3 and qs * primes[j] <=
math.isqrt(original_n):
            qs *= primes[j]
            j += 1

        if qs == 1:
            break

        gcd_result, _, _ = extended_gcd(n, qs)
        d = gcd_result
        if d > 1:
            for prime in primes:
                if prime > d:
                    break

```

```

        while d % prime == 0:
            factors.append(prime)
            d //= prime
        n //= (qs // d) if d > 1 else qs
        i = j
    else:
        i += 1

if n > 1:
    factors.append(n)

if len(factors) == 2:
    return sorted(factors)
elif len(factors) > 2:
    p = factors[0]
    q = 1
    for f in factors[1:]:
        q *= f
    return [p, q]
else:
    return factors

def factorize_n(n):
    print(f"[Факторизация n = {n}]")
    factors = trial_division_factorization(n)
    if len(factors) == 2:
        p, q = factors[0], factors[1]
        print(f"  Найдены простые множители: p = {p}, q = {q}")
        return p, q
    else:
        print(f"  Факторы: {factors}")
        raise ValueError("Не удалось разложить n на два простых множителя")

def main():
    print("="*60)
    print("ЛАБОРАТОРНАЯ РАБОТА №5: КРИПТОАНАЛИЗ ШИФРА RSA")
    print("="*60)
    print("Метод: факторизация модуля n (пробные деления)")
    print("="*60)

    print("\nВВЕДИТЕ ДАННЫЕ ДЛЯ КРИПТОАНАЛИЗА")
    print("(Пример: e=131, n=21733, C=258)")
    print("-" * 40)

    try:
        e = int(input("Введите e (открытая экспонента): "))
        n = int(input("Введите n (модуль): "))
        C = int(input("Введите C (зашифрованное сообщение): "))
    except ValueError:
        print("Ошибка: введите целые числа.")
        return

    print("\n" + "="*60)
    print("[Исходные данные]")
    print(f"  Открытый ключ: (e={e}, n={n})")
    print(f"  Зашифрованное сообщение: C = {C}")
    print("="*60)

    try:
        p, q = factorize_n(n)
    except ValueError as ve:

```

```

        print(f" Ошибка: {ve}")
        return

phi = (p - 1) * (q - 1)
print(f"\n[Вычисление закрытого ключа]")
print(f"  $\phi(n) = (p-1)*(q-1) = \{phi\}$ ")

try:
    d = mod_inverse(e, phi)
    print(f" Закрытый ключ d = {d}")
    print(f" Проверка:  $e*d \bmod \phi(n) = \{(e * d) \% phi\}$ ")
except ValueError as ve:
    print(f" Ошибка при вычислении d: {ve}")
    return

M = bin_pow(C, d, n)
print(f"\n[Расшифрование]")
print(f"  $M = C^d \bmod n = \{C\}^{\{d\}} \bmod \{n\} = \{M\}$ ")
print(f" Расшифрованное число: M = {M}")

C_check = bin_pow(M, e, n)
print(f" Проверка:  $M^e \bmod n = \{M\}^{\{e\}} \bmod \{n\} = \{C\_check\}$ ")
if C_check == C:
    print(" ✓ Проверка пройдена успешно!")
else:
    print(" ✗ Ошибка: проверка не пройдена!")

result = f"""\Результаты криптоанализа RSA
{'='*50}
Исходные данные:
Открытый ключ: e = {e}, n = {n}
Зашифрованное сообщение: C = {C}

Факторизация модуля n:
p = {p}
q = {q}
n = {p} * {q} = {p*q}

Вычисление закрытого ключа:
 $\phi(n) = (p-1)*(q-1) = \{phi\}$ 
 $d = e^{-1} \bmod \phi(n) = \{d\}$ 
Проверка:  $e*d \bmod \phi(n) = \{(e * d) \% phi\}$ 

Расшифрование:
 $M = C^d \bmod n = \{C\}^{\{d\}} \bmod \{n\} = \{M\}$ 

Проверка:
 $M^e \bmod n = \{M\}^{\{e\}} \bmod \{n\} = \{C\_check\}$ 
{'✓ Проверка пройдена' if C_check == C else '✗ Проверка не пройдена'}
"""
print("\n" + result)

save = input("Сохранить результат в файл? (y/n): ").lower().strip()
if save == 'y':
    filename = input("Введите имя файла: ")
    if not filename.endswith('.txt'):
        filename += '.txt'
    with open(filename, 'w', encoding='utf-8') as f:
        f.write(result)
    print(f"Результаты сохранены в файл: {filename}")

```

```
if __name__ == "__main__":  
    main()
```